

SAM 2图像分割模型微调指南：仅需60行代码

2024年08月14日 由 alex 发表

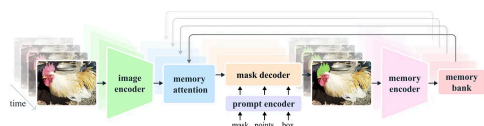
1405

0

SAM2 (Segment Anything 2) 是 Meta 公司推出的一个新模型，旨在对图像中的任何内容进行分割，而不局限于特定的类别或领域。该模型的独特之处在于其训练数据的规模：1100 万张图像和 110 亿个掩码。这种广泛的训练使 SAM2 成为训练新图像分割任务的强大起点。

你可能会问，既然 SAM 可以分割任何图像，为什么我们还要重新训练它呢？答案是，SAM 擅长常见对象，但在罕见或特定领域的任务上表现不佳。不过，即使在 SAM 的结果不够理想的情况下，也可以通过在新数据上对其进行微调来显著提高模型的能力。在很多情况下，这比从头开始训练一个模型所需的训练数据更少，效果更好。

本文演示了如何在新数据上对 SAM2 进行微调，只需 60 行代码（不包括注释和导入）。



Segment Anything 的工作原理

SAM 的主要工作方式是获取图像和图像中的一个点，然后预测包含该点的段的掩码。这种方法无需人工干预即可实

现完整的图像分割，而且对分割的类别或类型没有限制。

使用 SAM 进行完整图像分割的步骤：

1. 在图像中选择一组点
2. 使用 SAM 预测包含每个点的片段
3. 将得到的片段合并为一张地图

虽然 SAM 也可以利用遮罩或边界框等其他输入，但这些主要与涉及人工输入的交互式分割相关。在本教程中，我们将专注于全自动分割，并只考虑单点输入。

下载 SAM2 并设置环境

请按照安装说明进行操作。

一般来说，你需要 Python ≥ 3.11 和 PyTorch。

此外，我们还将使用 OpenCV，可以使用以下方法安装：

```
pip install opencv-python
```

下载预训练模型

你还需要从以下网址下载预训练模型：

<https://github.com/facebookresearch/segment-anything-2?tab=readme-ov-file#download-checkpoints>

有多种模型可供选择，均与本教程兼容。我建议使用小型模型，它的训练速度最快。

下载训练数据

下一步是下载用于微调模型的数据集。在本教程中，我们将使用 LabPics1 数据集来分割材料和液体。你可以从以下网址下载数据集：

<https://zenodo.org/records/3697452/files/LabPicsV1.zip?download=1>

准备数据阅读器

我们首先需要编写的是数据阅读器。它将为网络读取和准备数据。

数据阅读器需要生成

1. 图像
2. 图像中所有片段的掩码。
3. 每个遮罩内的随机点

让我们从加载依赖项开始：

```
import numpy as np
import torch
import cv2
import os
from sam2.build_sam import
from sam2.sam2_image_predictor
```

接下来，我们列出数据集中的所有图像：

```
data_dir=r"LabPicsV1/" # Path to the dataset
data=[] # list of files in the dataset
for ff, name in enumerate(os.listdir(data_dir)):
    data.append({"image":data_dir+name,
```

现在是加载训练批次的主函数。训练批次包括 一幅随机图像、属于这幅图像的所有分割蒙版，以及每个蒙版中的一个随机点：

```
def read_batch(data): # read a batch of data
    # select image
```

```

ent = data[np.random.randint(0, len(data))]
Img = cv2.imread(ent["image"])
ann_map = cv2.imread(ent["ann_map"])
# resize image
r = np.min([1024 /
            Img.shape[0], 1024 /
            Img.shape[1]])
Img = cv2.resize(Img, None, fx=r, fy=r)
ann_map = cv2.resize(ann_map, None, fx=r, fy=r)
# merge vessels and material maps
mat_map = ann_map[:, :, 0]
ves_map = ann_map[:, :, 1]
mat_map[mat_map==0] = 255
# Get binary masks and points
inds = np.unique(mat_map)
points = []
masks = []
for ind in inds:
    mask = (mat_map == ind)
    masks.append(mask)
    coords = np.argwhere(mask)
    yx = np.array(coords).T
    points.append(yx)
return Img, np.array(points)

```

该功能的第一部分是随机选择一张图片并加载它：

```

ent = data[np.random.randint(0, len(data))]
Img = cv2.imread(ent["image"])
ann_map = cv2.imread(ent["ann_map"])
Note that OpenCV reads images in BGR format.

```

请注意，OpenCV 以 BGR 格式读取图像，而 SAM 希望读取 RGB 图像。通过使用 [...::-1]，我们将图像从 BGR 变为 RGB。

SAM 希望图像大小不超过 1024，因此我们将调整图像和注释映射的大小。

```

r = np.min([1024 / Img.shape[0], 1024 / Img.shape[1]])
Img = cv2.resize(Img, (int(Img.shape[0]*r), int(Img.shape[1]*r)))
ann_map = cv2.resize(ann_map, (int(Img.shape[0]*r), int(Img.shape[1]*r)))

```

重要的一点是，在调整标注图 (ann_map) 的大小时，我们使用 INTER_NEAREST 模式 (近邻)。在标注图中，每个像素值都是其所属图段的索引。因此，使用不会给地图带来新值的调整大小方法非常重要。

下一个区块是 LabPics1 数据集的特定格式。注释图 (ann_map) 包含一个通道的图像血管分割图和另一个通道的材料注释图。我们将把它们合并为一张地图。

```

mat_map = ann_map[:, :, 0]
ves_map = ann_map[:, :, 2]
mat_map[mat_map==0] = ves_map

```

这样我们就得到了一个映射图 (mat_map)，其中每个像素的值都是它所属片段的索引 (例如：所有值为 3 的单元格都属于片段 3)。我们希望将其转换为一组二进制掩码 (0/1)，其中每个掩码对应一个不同的片段。此外，我们希望从每个掩码中提取一个点。

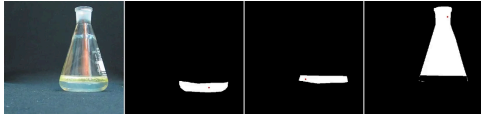
```

inds = np.unique(mat_map)[1:]
points = [] # list of all points
masks = [] # list of all masks
for ind in inds:
    mask = (mat_map == ind).astype(np.uint8)
    masks.append(mask)
    coords = np.argwhere(mask)
    yx = np.array(coords).T

```

```
        points.append([
return Img,np.array(masks),
```

就是这样！我们得到了图像 (img)、与图像中的片段相对应的二进制掩码列表 (掩码)，以及每个掩码中一个点的坐标 (点)。



加载 SAM 模型

现在让我们加载网络：

```
sam2_checkpoint = "sam2_hie
model_cfg = "sam2_hiera.s.y
sam2_model = build_sam2(moc
predictor = SAM2ImagePredic
```

任何分段 一般结构

在设置训练参数之前，我们需要了解 SAM 模型的基本结构。

SAM 由三部分组成：

1) 图像编码器；2) 提示编码器；3) 掩码解码器。

图像编码器负责处理图像并创建代表图像的嵌入。这部分由一个 ViT 变换器组成，是网络中最大的组成部分。我们通常不想对它进行训练，因为它已经给出了很好的表示，而训练将需要大量资源。

提示编码器处理网络的额外输入，在我们的例子中就是输入点。

掩码解码器接收图像编码器和提示编码器的输出，生成最终的分割掩码。一般来说，我们只想训练掩码解码器，或许还想训练提示编码器。这些部分都是轻量级的，可以通过适度的 GPU 进行快速微调。

设置训练参数：

我们可以通过设置来启用掩码解码器和提示编码器的训练：

predictor.model.sam_mask_de
predictor.model.sam_prompt_

接下来，我们定义标准的 adamW 优化器：

```
optimizer=torch.optim.AdamV
```

我们还将使用混合精度训练，这是一种更节省内存的训练策略：

```
scaler = torch.cuda.amp.GradScaler()
```

主训练循环

现在让我们构建主训练循环。第一部分是读取和准备数据：

```
for itr in range(100000):
    with torch.cuda.amp.autocast():
        image, mask, input = next(data_loader)
        if mask.shape[0] > 0:
            predictor.set_input(image, mask, input)
```

首先，我们对数据进行混合精度处理，以提高训练效率：

```
with torch.cuda.amp.autocast
```

接下来，我们使用之前创建的阅读器函数来读取训练数据：

```
image,mask,input_point, inp
```

我们将加载的图像通过图像编码器（网络的第一部分）：

```
predictor.set_image(image)
```

接下来，我们使用网络提示编码器处理输入点：

```
mask_input, unnorm_coords  
sparse_embeddings, dense_
```

请注意，在这一部分我们还可以输入方框或掩码，但我们不会使用这些选项。

现在，我们已经对提示（点）和图像进行了编码，最后就可以预测分割掩码了：

```
batched_mode = unnorm_coords  
high_res_features = [feat_1  
low_res_masks, prd_scores,  
prd_masks = predictor._tr
```


该代码的主要部分是 `model.sam_mask_decoder`，它运行网络的掩码解码器部分，并生成分割掩码 (`low_res_masks`) 及其分数 (`prd_scores`)。

这些掩码的分辨率低于原始输入图像，并在后处理掩码函数中被调整为原始输入图像的大小。

这样我们就能得到网络的最终预测结果：`prd_masks`包含每个输入点的 3 个预测掩码，但我们只使用每个点的第一个掩码。`prd_scores` 包含网络认为每个掩码有多好的分数（或它对预测有多确定）。

损失函数

分割损失

有了网络预测结果，我们就可以计算损失了。首先，我们计算分割损失，即预测的掩码与地面真实掩码相比有多好。为此，我们使用标准的交叉熵损失。

首先，我们需要使用 `sigmoid` 函数将预测掩码 (`prd_mask`) 从对数转换为概率：

```
prd_mask = torch.sigmoid(prd_mask)
```

接下来，我们将地面实况掩码转换为 `torch` 张量：

```
prd_mask = torch.sigmoid(prd_mask)
```

最后，我们使用地面实况 (`gt_mask`) 和预测概率图 (`prd_mask`) 手动计算交叉熵损失 (`seg_loss`)：

```
seg_loss = (-gt_mask * torch
```

(我们添加 0.0001 是为了防止对数函数因数值为零而爆炸)。

分数损失 (可选)

除了掩码之外，网络还会预测每个预测掩码的得分。这部分的训练不那么重要，但也很有用。要训练这一部分，我们首先需要知道每个预测掩码的真实得分。也就是说，预测的掩码实际有多好。我们将使用交集大于联合 (IOU) 指标来比较 GT 掩膜和相应的预测掩膜。IOU 简单来说就是两个掩码的重叠部分除以两个掩码的合并面积。首先，我们计算预测掩膜和 GT 掩膜之间的交集 (两者重叠的区域)：

```
inter = (gt_mask * (prd_mas
```

我们使用阈值 ($\text{prd_mask} > 0.5$) 将预测掩码从概率掩码转换为二进制掩码。

接下来，我们用交集除以预测掩码和 gt 掩码的合并区域 (联合)，得到 IOU：

```
iou = inter / (gt_mask.sum(
```

我们将把 IOU 作为每个面具的真实得分，并把预测得分与我们刚刚计算出的 IOU 之间的绝对差值作为得分损失。

```
score_loss = torch.abs(pred_
```

最后，我们合并了分割损失和分数损失（前者的权重更高）：

```
loss = seg_loss+score_loss*
```

最后一步：反向传播和保存模型

一旦我们得到了损失，接下来的步骤都是完全标准的。我们使用之前创建的优化器来计算反向传播，并更新权重：

```
predictor.model.zero_grad()  
scaler.scale(loss).backward()  
scaler.step(optimizer)  
scaler.update() # Mix preci
```

我们还希望在每1000个步骤后保存训练好的模型：

```
if itr==0: torch.save(pre
```

由于我们已经计算了IOU，我们可以将其显示为移动平均值，以便看到模型预测随着时间的推移的改进情况：

```
if itr==0: mean_iou=0  
mean_iou = mean_iou * 0.99  
print("step)",itr, "Accurac
```

现在，我们已经在不到60行的代码中完成了Segment-Anything 2的训练/微调（不包括注释和导入）。大约经过25000个步骤，你应该会看到主要的改进。

该模型将保存为“model.torch”。

推理：加载和使用经过训练的模型：

现在我们已经对模型进行了微调，让我们使用它来分割一张图片。

我们将按照以下步骤进行操作：

1. 加载我们刚刚训练的模型。
2. 为模型提供一张图像和一系列随机点。对于每个点，模型将预测包含该点和一个分数的段落掩模。
3. 将这些掩模拼接在一起，形成一个分割图。

首先，我们加载依赖项并将权重转换为float16，这样可以使模型运行速度更快（仅适用于推理）：

```
# use bfloat16 for the entire model
torch.autocast(device_type='cuda', dtype=torch.bfloat16)
```

接下来，我们加载一张示例图像和我们想要分割的图像区域的掩模（下载图像/掩模）：

```
image_path = r"sample_image.jpg"
mask_path = r"sample_mask.png"

def read_image(image_path, mask_path):
    img = cv2.imread(image_path)
    mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

    # Resize image to match mask dimensions
```

```

        r = np.min([1024 /
img = cv2.resize(in
mask = cv2.resize(n
        return img, mask
image,mask = read_image(imc

```

在我们要分割的区域内，随机选取30个样本点：

```

num_samples = 30 # number c
def get_points(mask,num_poi
    points=[]
    for i in range(num_
        coords = np.arg
        yx = np.array(c
        points.append([
    return np.array(poi
input_points = get_points(n

```

首先，加载标准的SAM模型（与训练中的模型相同）：

```

# Load model you need to h
sam2_checkpoint = "sam2_hie
model_cfg = "sam2_hiera.s.y
sam2_model = build_sam2(moc
predictor = SAM2ImagePredic

```

接下来，加载我们刚刚训练的模型的权重（model.torch）：

```

predictor.model.load_state_

```

运行经过微调的模型，预测我们选择的每个点的掩模：

```
with torch.no_grad(): # pre
    predictor.set_image
    masks, scores, logi
    point_coords=ir
    point_labels=np
)
```

现在我们有一系列预测的掩模及其得分。我们希望将它们拼接成一个一致的分割图。然而，许多掩模可能重叠，并且可能彼此不一致。由于我们是随机选择的点，所以很可能某些点会落在同一个段落内。

拼接的方法很简单，我们将根据它们的预测得分对预测的掩模进行排序：

```
np_masks = np.array(masks[:
np_scores = scores[:,0].flo
shorted_masks = np_masks[np
```

现在让我们创建一个空的分割图和一个占用图：

```
seg_map = np.zeros_like(sho
occupancy_mask = np.zeros_l
```

接下来，我们逐个将掩模（从高分到低分）添加到分割图中。只有在要添加的掩模与先前添加的掩模保持一致时，我们才会添加该掩模。这意味着只有当我

们要添加的掩模与已占用区域的重叠度小于15%时，我们才会添加它。

```
for i in range(shorted_mask):
    mask = shorted_masks[i]
    if (mask*occupancy_mask > 0.15):
        mask[occupancy_mask==0] = 0
        seg_map[mask] = i+1
        occupancy_mask[mask] = 1
```

就是这样了。

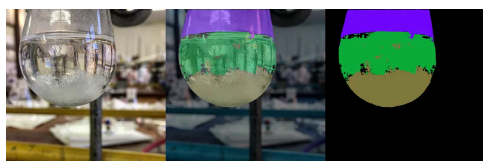
seg_mask现在包含了预测的分割图，每个段落都有不同的值，背景为0。

我们可以使用以下代码将其转换成彩色图：

```
rgb_image = np.zeros((seg_map.shape[0], seg_map.shape[1], 3))
for id_class in range(1, seg_map.max()+1):
    rgb_image[seg_map == id_class] = color_map[id_class]
```

并显示：

```
cv2.imshow("annotation", rgb_image)
cv2.imshow("mix", (rgb_image * 0.5 + image * 0.5))
cv2.imshow("image", image)
cv2.waitKey()
```



文章来源: <https://medium.com/@sagieppe>
l/train-fine-tune-segment-anything-2-sa
m-2-in-60-lines-of-code-928dd29a63b3

标签: [机器学习](#) [计算机视觉](#) [人工智能](#) 0 评论



欢迎关注ATYUN
官方公众号
商务合作及内容
投稿请联系邮箱:

上一篇 **【指南】**
微调情绪
分析模型
下一篇 使用Mifl
ow构建
与算法无
关的模型

评论

要发表评论，您必须先 [登录](#) 。

热门标签

行业 学习 机器学习 人工智能 公司板 人工智能未来 机器人 视觉识别
人工智能应用 谷歌 智能设备 人工智能教程 深度学习 自动驾驶 计算机视觉 医疗
语音识别 开源项目 神经网络 图像识别



ATYUN.COM 官网
人工智能教程资讯全方位服务平台

AI资讯	资源	常用工具	服务
行业资讯	AI MAP	AI写作工具	EVA
机器学习	数据模型	AI编程工具	智能HR
计算机视觉	数据集	AI设计工具	公司版
自然语言处理		AI视频工具	招聘
			联系我们



ATYUN公众平台

[友情链接](#)

[人工智能](#) [人工智能创业](#) [EVA](#) [智能HR](#)

© 2016 ATYUN.COM – AI TECH YUN